

MongoDB Assignment

Event Management & Ticketing System

Student Name: Lakshay Ghai
Subject: MongoDB
Submission Date: 19 January 2026

1. Introduction

This assignment demonstrates the design and logic of a MongoDB-based Event Management and Ticketing System. The system allows organizers to create events, users to browse them, and attendees to book tickets. The objective is to showcase understanding of MongoDB concepts including schema design, CRUD operations, aggregation pipelines, and indexing.

2. Database Design Overview

The database is designed using multiple collections. Each collection represents a real-world entity such as users, events, tickets, and categories. Logical relationships are maintained using reference fields.

3. Users Collection

The Users collection stores details of all users. Users can either be organizers or attendees.

```
{
  userId: "USR001",
  name: "Aarav Sharma",
  email: "aarav.sharma@gmail.com",
  phone: "9876543210",
  role: "organizer",
  createdAt: new Date()
}
```

4. Categories Collection

The Categories collection is used to classify events. Categories help in filtering and aggregation.

```
{
  categoryId: "CAT02",
  name: "Technology",
  description: "Technology conferences and workshops"
}
```

5. Events Collection

The Events collection stores event details created by organizers. Each event belongs to a category and includes ticket pricing and availability.

```
{
  eventId: "EVT001",
  title: "Indie Music Night",
  category: "Music",
  ticketPrice: 150,
  availability: 50
}
```

```

venue: "Delhi Convention Centre",
organizerId: "USR001",
price: 1500,
totalTickets: 300,
availableTickets: 180,
status: "upcoming"
}

```

6. Tickets Collection

The Tickets collection represents ticket bookings made by users for events.

```

{
  ticketId: "TKT001",
  eventId: "EVT001",
  userId: "USR002",
  quantity: 2,
  totalAmount: 3000,
  status: "booked"
}

```

7. CRUD Operations

CRUD operations demonstrate how data is created, read, updated, and deleted.

```

// Create
db.events.insertOne({ title: "Startup Meetup" });

// Read
db.events.find({ status: "upcoming" });

// Update
db.events.updateOne(
  { eventId: "EVT001" },
  { $inc: { availableTickets: -1 } }
);

// Delete
db.events.deleteOne({ status: "cancelled" });

```

8. Aggregation Pipelines

Aggregation pipelines are used to generate analytical reports.

```

// Top 5 Events by Ticket Sales
db.tickets.aggregate([
  { $group: { _id: "$eventId", ticketsSold: { $sum: "$quantity" } } },
  { $sort: { ticketsSold: -1 } },
  { $limit: 5 }
]);

// Total Revenue Earned by Organizer
db.tickets.aggregate([
  {
    $lookup: {
      from: "events",
      localField: "eventId",
      foreignField: "eventId",
      as: "eventDetails"
    }
  },
  { $unwind: "$eventDetails" },
  {
    $group: {
      _id: "$eventDetails.organizerId",
      totalRevenue: { $sum: "$totalAmount" }
    }
  }
]);

```

```
    }  
  }  
  });
```

9. Indexing

Indexes are created to improve query performance.

```
db.events.createIndex({ dateTime: 1 });  
db.events.createIndex({ category: 1 });  
db.tickets.createIndex({ userId: 1, eventId: 1 });
```

10. Conclusion

This assignment successfully demonstrates MongoDB schema design, sample data modeling, CRUD operations, aggregation pipelines, and indexing. The project reflects a clear understanding of MongoDB concepts.

Note: The data and queries included in this assignment are created for academic and demonstration purposes.